

Open source, yet proprietary

FreeRTOS has been developed in such a way that it forms the core of the software. Although RTOS kernel is open source, application codes that use this kernel still remain proprietary and are closed source. It has both free and commercial support forums that are actively run by engineers and engineering companies all over the Internet.

Although many open source electronics software end up costing a lot, FreeRTOS is available free of cost on the Internet for download, deployment and experimentation.

In the case of open source software, the source code gets fragmented with the availability of any new version, whereas FreeRTOS follows a very controlled release procedure where all its official ports are updated simultaneously.

Many free tools do not have any life; these are either trial versions that last only 30 days or are student versions with limited usage, while FreeRTOS is a forever software that comes at no cost at all.

A commercially-licensed version of FreeRTOS is also available with Real Time Engineers Ltd.

Features it supports

In order to understand the benefits of using FreeRTOS it is very important to understand its features in detail from the system's memory that it occupies to the different architectures that it supports. But when you speak about memory, it is important to talk about both random access memory (RAM) and read only memory (ROM).

The size of RAM that FreeRTOS occupies depends on the application it is being used for. When used with IAR STR71X ARM7 port with four priorities, the scheduler in FreeRTOS uses 236 bytes, the queues created use 76 bytes plus the storage area of the queue and the total tasks created use 64 bytes (four characters for each task created multiplied by the total number of tasks, that is, four) plus the stack size of each task.

On the contrary, the ROM usage

Using FreeRTOS with an MCU

FreeRTOS easily runs on MCUs with different brands and different architectures, but you will find slight variations in the steps for each. Although www.freertos.org lists procedures for working with FreeRTOS on many different MCUs that officially support FreeRTOS, described below are the steps for using it on Atmel ATSAM20 with Atmel Studio.

ATSAMD20 is an MCU by Atmel with ARM Cortex-M0+ architecture. Atmel Studio integrated development environment (IDE) uses the visual studio framework and FreeRTOS plugin to work with SAMD20 Xplained Pro that contains ATSAM20 MCU.

1. Open the following file in Atmel Studio IDE: `FreeRTOS/Demo/CORTEX_M0+_Atmel_SAMD20_Xplained/RTOSDemo.atsh`
2. Locate definition `mainCREATE_SIMPLE_BLINKY_DEMO_ONLY` at the top of `main.c`
3. Set `mainCREATE_SIMPLE_BLINKY_DEMO_ONLY` to 1 to create the simple blinky demo, or to 0 to create the comprehensive demo that also includes the command line interpreter.
4. Select Rebuild RTOSDemo from Atmel Studio Build menu (or press F7) to build the demo project.
5. Connect a USB cable between the USB port on SAMD20 Xplained Pro board and the host computer you are using for this operation.
6. In order to program the flash memory of the MCU you are using, select Start Debugging and Break from Atmel Studio Debug menu and start the debugging session.

by FreeRTOS depends on the compiler, architecture and RTOS kernel configuration, which alone requires about 5kB to 10kB of ROM.

It follows a fixed coding standard that determines that no non-defined task gets executed during an interrupt or in a critical situation.

Users say...

If you go online and look up FreeRTOS, you will realise that majority of people appreciate both its features and developers, Real Time Engineers Ltd.

Taking an example from sourceforge.net, almost 95 per cent of users seem to have experienced that it is lightweight and that its architecture makes it an independent OS for embedded systems. They also acknowledge the fact that this OS supports all types of platforms and does not crash even when being used by beginners for the first time.

On the other hand, if you visit www.edaboard.com, you will find beginners who wish to learn and understand FreeRTOS and other RTOSes only to understand the difference

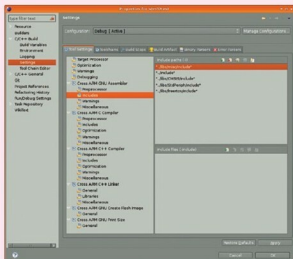


Fig. 5: FreeRTOS with STM32F4 MCU using Linux platform

between a general OS and a real-time one, and to use it with different MCUs easily.

Additionally, you will also find some beginners who are studying FreeRTOS through different documents available online and are using it with various MCUs.

Some engineers at electronics.stackexchange.com have compared FreeRTOS with other RTOSes to try and prove its popularity over others by pointing out the high number of downloads of it and its extremely tiny footprint of about 4kB. ●

The author is a technical correspondent at EFY